

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning (BL 5)

Assessment Tool Weightage: 20%

QUESTIONS: 15

Total Marks: 25

Annexure A

Industry Problem Solving

Code of Conduct:

I V.Madhulika (192312620) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

V.Madhulika

Signature of the student

Q1. A retail company wants to recommend products to customers based on past purchases. Design a solution using K-Nearest Neighbour or Case-Based Learning. Explain how similarity is measured, how recommendations are generated, and how the system adapts to new customer data.

Answer:

A K-Nearest Neighbour based collaborative-filtering recommender is well suited here. Each customer is represented as a vector of purchase frequencies or ratings across the product catalogue (a user-item matrix). Similarity between customers is measured using cosine similarity or Euclidean distance on these vectors, so two customers who buy similar baskets of products end up close together regardless of raw purchase volume. To generate recommendations for a target customer, the system finds the K most similar past customers (nearest neighbours), aggregates the products they purchased that the target customer has not yet bought, and ranks them by how frequently/highly they were purchased/rated among the neighbours. Item-based KNN (finding products similar to what the customer already bought) is often preferred at retail scale because item similarity is more stable over time than user similarity. As new purchases arrive, the user-item matrix is incrementally updated and neighbour sets are recomputed (or approximated using periodic batch re-indexing with approximate nearest-neighbour structures like KD-trees or locality-sensitive hashing) so the system adapts without a full retrain, keeping recommendations current with evolving customer behaviour.

Q2. A manufacturing industry needs to predict machine output based on sensor readings that vary over time. Propose a solution using Locally Weighted Regression. Explain how local models improve prediction accuracy and how the system handles noisy or incomplete data.

Answer:

Locally Weighted Regression (LWR) is appropriate because the relationship between sensor readings (temperature, vibration, pressure, load) and machine output is unlikely to be a single global linear function — it typically shifts across different operating regimes (start-up, steady-state, high-load). For each new query point (current sensor reading), LWR fits a small linear regression using only the nearby historical readings, weighted by a Gaussian kernel based on distance in sensor-feature space, rather than fitting one fixed model for the whole operating range. This lets the model automatically adapt its behaviour to the local regime the machine is currently in, improving accuracy over a single global regression that would average across very different conditions. To handle noisy sensor data, the kernel bandwidth can be widened slightly so that individual noisy readings have less influence, and outlier readings can be down-weighted or filtered using a robust weighting scheme (e.g. capping extreme residuals). Missing or incomplete readings are handled by imputing from the nearest available sensor readings in time, or by simply excluding the missing feature from the distance calculation for that query so the local model is still computed from whatever sensor values are available.

Q3. A healthcare company aims to classify patients into risk categories using historical medical data. Suggest a model using KNN or RBF networks. Explain how the algorithm handles large datasets, feature selection, and accuracy improvement in real-world scenarios.

Answer:

An RBF network is a strong choice for patient risk classification because it can model the non-linear relationships that typically exist between clinical measurements (age, blood pressure, cholesterol, glucose levels) and risk category, while remaining faster at prediction time than KNN once trained. Gaussian centres are placed at representative points in the clinical feature space (found via clustering, e.g. K-Means, on the training data), and each patient's risk score is computed as a weighted combination

of Gaussian activations measuring how close the patient's profile is to each centre. For large datasets, the number of centres is kept much smaller than the number of patients (unlike KNN, which must compare against every stored patient at prediction time), which keeps inference fast and scalable. Feature selection is important because irrelevant clinical variables dilute the distance calculation in the Gaussian kernels; techniques such as correlation analysis, mutual information, or wrapper methods (e.g. recursive feature elimination) are used beforehand to keep only clinically meaningful predictors. In practice, accuracy is improved by normalising features to comparable scales, tuning the number of centres and kernel width via cross-validation, and combining the RBF network's probability outputs with clinical guideline thresholds so borderline predictions are flagged for physician review rather than acted on automatically.

Q4. An e-commerce platform wants to detect similar products for dynamic pricing strategies. Design a solution using Case-Based Learning. Discuss how similarity metrics are defined, how past cases are reused, and how the system scales with increasing product data.

Answer:

Each product is stored as a 'case' consisting of its attributes (category, brand, specifications, historical demand, seasonal sales pattern) together with the pricing decisions and outcomes that were previously applied to it or to comparable products. When a pricing decision is needed for a product, the system computes similarity against the case base using a weighted combination of categorical similarity (matching category/brand) and numerical similarity (specification and price-range distance), giving more weight to attributes known to influence price sensitivity. The most similar past cases are retrieved, and their successful pricing strategies (discount levels, price elasticity response) are reused and adapted — for example scaled to the new product's price range — to propose a dynamic price for the current product. The outcome of each new pricing decision (change in sales volume, revenue) is then stored back into the case base as a new case, so the system continuously learns from real outcomes (a full case-based reasoning cycle of retrieve-reuse-revise-retain). To scale as the product catalogue grows, cases are indexed by category and organised into a hierarchical or clustered case base so that similarity search only needs to examine a relevant subset of cases rather than the entire catalogue, and approximate nearest-neighbour indexing is used to keep retrieval fast even with millions of products.

Q5. A logistics company needs to forecast delivery times based on location and traffic patterns. Propose a solution using Radial Basis Function networks or Locally Weighted Regression. Explain how the model captures non-linear relationships and improves prediction reliability under varying conditions.

Answer:

Locally Weighted Regression is well suited to delivery-time forecasting because traffic conditions and delivery times vary non-linearly and locally by route, time of day, and location — a delivery through a busy city centre at rush hour behaves very differently from a suburban delivery at midday, and a single global model would blur these distinct regimes together. For a new delivery request, LWR weights historical deliveries by their similarity in feature space (route segment, distance, time of day, day of week, current traffic index) using a kernel function, then fits a small local regression on just those similar past deliveries to predict the expected time. This captures non-linear relationships automatically, since different regions of the feature space get their own locally-fitted model rather than being forced into one global linear trend. Prediction reliability under varying conditions is improved by including traffic and weather features directly in the distance metric so that the 'similar' historical deliveries used for weighting genuinely reflect comparable conditions, and by periodically refreshing the training data with recent deliveries so that the local models reflect current traffic patterns rather than stale historical averages.

Q6. A banking system wants to detect potential loan defaulters based on customer financial history and transaction patterns. Design a solution using K-Nearest Neighbour or Logistic Regression. Explain how similarity or decision boundaries are used, how features are selected, and how the system adapts to new customer data over time.

Answer:

Logistic Regression is the stronger choice for loan default prediction because banking regulators typically require interpretable, auditable models with a clear decision boundary and coefficients that can be explained to customers and regulators (unlike KNN's opaque neighbour-voting). The model learns a linear decision boundary in the space of financial features (income, debt-to-income ratio, credit history length, repayment history, transaction volatility) and outputs a probability of default via the sigmoid function; applicants above a chosen probability threshold are flagged as high risk. Feature selection is critical and is typically done using domain knowledge combined with statistical techniques (information value / weight-of-evidence analysis common in credit scoring, correlation checks, and regularisation such as L1/Lasso which naturally zeroes out weak predictors) so the model is both accurate and compliant with fair-lending requirements. As new repayment and transaction data arrives, the model is periodically retrained on a rolling window of recent data (or updated incrementally using online gradient descent) so that its coefficients reflect current economic conditions and customer behaviour rather than becoming stale; the threshold and coefficients are also monitored over time for drift, with alerts triggered if the population's feature distribution shifts significantly from what the model was trained on.

Q7. A smart agriculture system aims to predict crop yield based on soil quality, weather conditions, and irrigation levels. Propose a solution using Locally Weighted Regression or RBF networks. Explain how local learning improves predictions and how the system handles seasonal variations and noisy environmental data.

Answer:

An RBF network is well suited to crop-yield prediction because the relationship between soil quality, weather, and irrigation and the resulting yield is highly non-linear and varies by season and crop stage. Gaussian centres are placed at representative combinations of soil-weather-irrigation conditions observed in historical data (for example clustered by growing season and region), and each hidden unit's activation measures how close the current field conditions are to one of these representative regimes; the output layer then linearly combines these activations to produce a yield estimate. This local learning approach lets the model represent, for instance, that high irrigation improves yield during a dry season but has little additional benefit during a wet season, without needing a single global formula to capture every interaction. Seasonal variation is handled by including time-of-year / growth-stage as an explicit feature in the centre space so that the network naturally learns season-specific response curves. Noisy sensor data (soil moisture sensors, rain gauges) is mitigated by smoothing readings over a short time window before they are fed into the network and by choosing a kernel width wide enough that individual noisy readings do not disproportionately activate a single centre, giving more stable yield predictions.

Q8. A cybersecurity firm wants to identify abnormal network behavior to detect intrusions. Design a system using KNN or Case-Based Learning. Explain how similarity between network patterns is measured, how anomalies are detected, and how the model updates with new attack patterns.

Answer:

A KNN-based anomaly detection approach represents each network session or traffic window as a feature vector (packet size distribution, connection duration, protocol type, byte counts, number of failed

logins, port access patterns). Similarity between sessions is measured using a distance metric such as Euclidean or Mahalanobis distance over normalised features, so sessions with comparable traffic characteristics cluster together. Anomalies are detected by computing, for each new session, its distance to its K nearest neighbours among known-normal traffic: if the average distance to those neighbours (or local outlier factor) exceeds a learned threshold, the session is flagged as anomalous because it does not resemble typical behaviour, without requiring the intrusion to match a known signature. This makes the approach effective against novel attack variants, not just previously catalogued ones. As new confirmed attack patterns are identified by security analysts, they are added to the reference dataset (either as new labelled 'attack' neighbours for a supervised KNN classifier, or as new normal baselines if behaviour patterns legitimately shift), and the reference set is periodically pruned and re-indexed so the nearest-neighbour search remains fast and the model's notion of 'normal' stays current as the network environment evolves.

Q9. A streaming platform aims to recommend movies to users based on viewing history and ratings. Propose a solution using KNN or Case-Based Learning. Explain how user similarity is calculated, how recommendations are generated, and how the system handles large-scale user data efficiently.

Answer:

A KNN-based collaborative filtering recommender represents each user as a vector of ratings (explicit ratings or implicit watch-time/completion signals) across movies. User similarity is calculated using cosine similarity or Pearson correlation on the co-rated movies between two users, which handles differing rating scales (some users rate generously, others harshly) better than raw Euclidean distance. To recommend movies for a target user, the system identifies the K most similar users and predicts the target user's likely rating for movies they have not seen as a similarity-weighted average of those neighbours' ratings, then returns the highest-scoring unseen titles. At the scale of a streaming platform, computing exact nearest neighbours across millions of users for every recommendation request is computationally prohibitive, so the system instead uses approximate nearest-neighbour techniques (locality-sensitive hashing or precomputed similarity clusters), item-based KNN (which is generally more stable and requires less frequent recomputation than user-based KNN because item-to-item similarity changes more slowly), and periodic offline batch jobs to precompute and cache neighbour lists, with only lightweight lookups needed at request time to keep recommendations responsive.

Q10. A transportation company wants to optimize route selection based on traffic, distance, and fuel consumption. Design a solution using RBF networks or Locally Weighted Regression. Explain how the model captures complex relationships between variables and improves decision-making in dynamic and real-time environments.

Answer:

An RBF network is proposed to estimate the expected cost (combining travel time, fuel consumption, and traffic delay) of candidate routes. Gaussian centres are placed at representative combinations of route features — distance, road type, historical congestion level, and time of day — learned from historical trip data (e.g. via K-Means clustering on past routes). For a candidate route, the network computes how strongly it activates each centre based on feature similarity, and combines these activations to estimate total cost; because the Gaussian activations are non-linear, the network naturally captures complex interactions, such as distance mattering much less than congestion level during rush hour but dominating cost during off-peak times, without needing these interactions to be manually specified. In a real-time setting, live traffic feed data is incorporated as an additional input feature so predictions reflect current conditions rather than only historical averages, and because the trained RBF network only requires a forward pass through a fixed set of centres (rather than searching the whole

historical dataset as KNN would), it can score multiple candidate routes quickly enough to support real-time route selection and re-routing when conditions change mid-journey.

Q11. A telecommunications company wants to recommend suitable data plans to customers based on their usage patterns. Design a solution using K-Nearest Neighbour or Case-Based Learning. Explain how similarity between users is measured, how recommendations are generated, and how the system adapts to changing user behavior over time.

Answer:

A KNN-based recommender represents each customer as a feature vector of usage patterns — average monthly data consumption, call minutes, SMS volume, peak usage times, and device type. Similarity between customers is measured using a normalised Euclidean or cosine distance across these usage features so that customers with comparable consumption habits are grouped together regardless of absolute scale differences. To recommend a data plan for a customer, the system finds their K nearest neighbours among customers whose current plan choice has resulted in high satisfaction (low overage charges, low unused-quota waste, few complaints), and recommends the plan(s) that were successful for the majority of those similar customers. Because usage habits change over time (e.g. increased streaming or remote-work data needs), the system recomputes each customer's feature vector on a rolling window (such as the last three months of usage) rather than using a static lifetime profile, so a customer whose behaviour shifts will naturally be compared against a different, more relevant set of neighbours and receive updated plan recommendations; periodic batch reprocessing keeps the neighbour index current as the overall customer base's usage patterns evolve.

Q12. An energy management company aims to predict electricity consumption based on factors such as time, weather, and appliance usage. Propose a solution using Locally Weighted Regression or RBF networks. Explain how local learning captures variations in consumption patterns and how the model handles noisy and seasonal data.

Answer:

Locally Weighted Regression is proposed because electricity consumption behaves very differently depending on the local context — weekday versus weekend, summer cooling load versus winter heating load, morning versus evening peaks — and a single global regression would average across these very different regimes and lose accuracy. For a prediction at a given time and set of weather/appliance conditions, LWR weights historical consumption readings by their similarity in the feature space (time of day, day of week, temperature, appliance load), giving the most influence to historically similar contexts, and fits a small local linear model from just those weighted neighbours. This lets the model implicitly learn separate local response curves for each regime (e.g. how temperature affects consumption in summer versus winter) without manually splitting the data into segments. Seasonal variation is handled by including calendar features (month, season indicator) directly in the distance calculation so that seasonally similar historical periods are weighted more heavily. Noisy meter readings are smoothed with a short moving average before being used as training points, and the kernel bandwidth is tuned so that isolated noisy spikes do not dominate any single local regression, improving the reliability of the forecast.

Q13. A fraud detection system in an online payment platform needs to classify transactions as legitimate or fraudulent. Design a solution using KNN or Logistic Regression. Explain how similarity or decision boundaries are used, how features are selected, and how the model adapts to evolving fraud patterns.

Answer:

Logistic Regression is a strong baseline for real-time fraud classification because it produces a fast, interpretable probability score that can be computed with minimal latency at transaction time, which is essential when a decision must be made within milliseconds during checkout. The model learns a decision boundary over transaction features — amount, merchant category, transaction velocity (number of transactions in a short window), location deviation from the customer's usual pattern, and device fingerprint changes — and outputs the probability that a transaction is fraudulent via the sigmoid function; transactions above a risk threshold are declined or routed to manual review. Feature selection combines domain expertise (known fraud indicators such as sudden high-value purchases or mismatched billing/shipping addresses) with statistical techniques such as L1 regularisation, which shrinks the weights of uninformative features toward zero and keeps the model both accurate and fast. Because fraud patterns evolve as fraudsters adapt to detection systems, the model is retrained frequently (daily or weekly) on the most recent confirmed fraud and legitimate transaction data, often supplemented by an ensemble or a KNN-based similarity check against recently confirmed fraud cases to catch fast-emerging attack patterns that the periodically-retrained logistic model has not yet learned.

Q14. A manufacturing company wants to detect defective products based on sensor readings and quality measurements. Propose a solution using RBF networks or Case-Based Learning. Explain how non-linear relationships are captured, how similarity between cases is measured, and how the system improves with additional production data.

Answer:

An RBF network is proposed for defect detection because the relationship between sensor readings (temperature, pressure, vibration, dimensional measurements) and defect probability is typically non-linear — a defect might only occur when several measurements simultaneously fall outside a normal joint range, rather than any single measurement crossing a fixed threshold. Gaussian centres are placed at representative 'normal' and 'defective' operating profiles identified from historical quality-control data, and the Gaussian activation for a new product's sensor readings measures how closely it matches each of these profiles; the output layer combines these activations into a defect probability. This captures complex, non-linear interactions between sensor readings automatically, since the network is not restricted to a single global linear boundary. Similarity to each centre is computed using the same Euclidean-distance-based Gaussian kernel used in training, so a product whose readings are far from all known-normal profiles produces low activation everywhere and is flagged as anomalous even if it doesn't closely resemble a previously seen defect type. As more production data is collected, the centres are periodically re-clustered and the network is retrained, allowing it to refine its notion of 'normal' and incorporate newly discovered defect signatures, steadily improving detection accuracy over the life of the production line.

Q15. A smart city system aims to predict traffic flow and congestion levels using data from sensors, weather, and time of day. Design a solution using Locally Weighted Regression or KNN. Explain how the model captures dynamic patterns, how predictions are made in real-time, and how scalability is maintained with large volumes of data.

Answer:

Locally Weighted Regression is proposed to predict congestion level at a given road segment because traffic behaviour is highly context-dependent — rush hour on a weekday behaves very differently from a weekend afternoon, and a single global model would smooth over these distinct dynamic patterns. For a prediction request, the system weights historical traffic observations by their similarity to the current context (road segment, time of day, day of week, weather condition) using a Gaussian kernel, and fits a small local regression on the most relevant historical observations, allowing the model to represent different traffic dynamics for different contexts without manually defining separate models for each. To

support real-time prediction across a city-wide sensor network, exact LWR (which requires solving a weighted regression per query) is made scalable by pre-partitioning the historical data by road segment and time-of-day bucket, so that each query only needs to search within its relevant partition rather than the entire city's historical dataset, and by using spatial indexing structures (KD-trees or grid indexes) to quickly retrieve the nearest historical observations. Sensor data streams are processed incrementally, with periodic batch jobs re-aggregating and pruning older data so that the reference dataset used for weighting stays a manageable, representative size even as the volume of incoming sensor data grows continuously.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	25	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand